Register no: 26MAG0064
Name: AADITYA SOGANI

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# - - - - - - - - - - - - - - - - - - -
# 1) Load and combine the two parquet files
# - - - - - - - - - - - - - - - - - - -
benign = pd.read_parquet(r"/mnt/Data/DataSet/Network Intrusion Detection/Benign-Monday-no-metadata.parquet")
dos = pd.read_parquet(r"/mnt/Data/DataSet/Network Intrusion Detection/DoS-Wednesday-no-metadata.parquet")

df = pd.concat([benign, dos], ignore_index=True)
print("Combined shape:", df.shape)
print(df["Label"].value_counts())
```

“””output: Combined shape: (1043822, 78)

Label	
Benign	850066
DoS Hulk	172846
DoS GoldenEye	10286
DoS slowloris	5385
DoS Slowhttptest	5228
Heartbleed	11
Name: count, dtype:	int64

```
# - - - - - - - - - - - - - - - - - - -
# 2) Separate features (X) from the label (y)
# - - - - - - - - - - - - - - - - - - -
# The Label column is categorical (Benign / DoS Hulk / etc.) -- it is NOT
# a feature to feed into PCA, but we keep it aside to color our scatter plot.
y = df["Label"]
numeric_df = df.drop(columns=["Label"]).select_dtypes(include=[np.number]).copy()

# Drop rows with missing values (none expected here, but kept for safety/consistency)
before = numeric_df.shape[0]
numeric_df = numeric_df.dropna()
y = y.loc[numeric_df.index]
print(f"Rows before dropna: {before}, after: {numeric_df.shape[0]}")
```

#output: Rows before dropna: 1043822, after: 1043822

```
# ------------------
# 3) Standardize the data
# ------------------
# PCA is variance-based, so features must be on the same scale first.
# Without this, high-magnitude features (e.g. Flow Duration in microseconds)
# would dominate the principal components purely due to units.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(numeric_df)

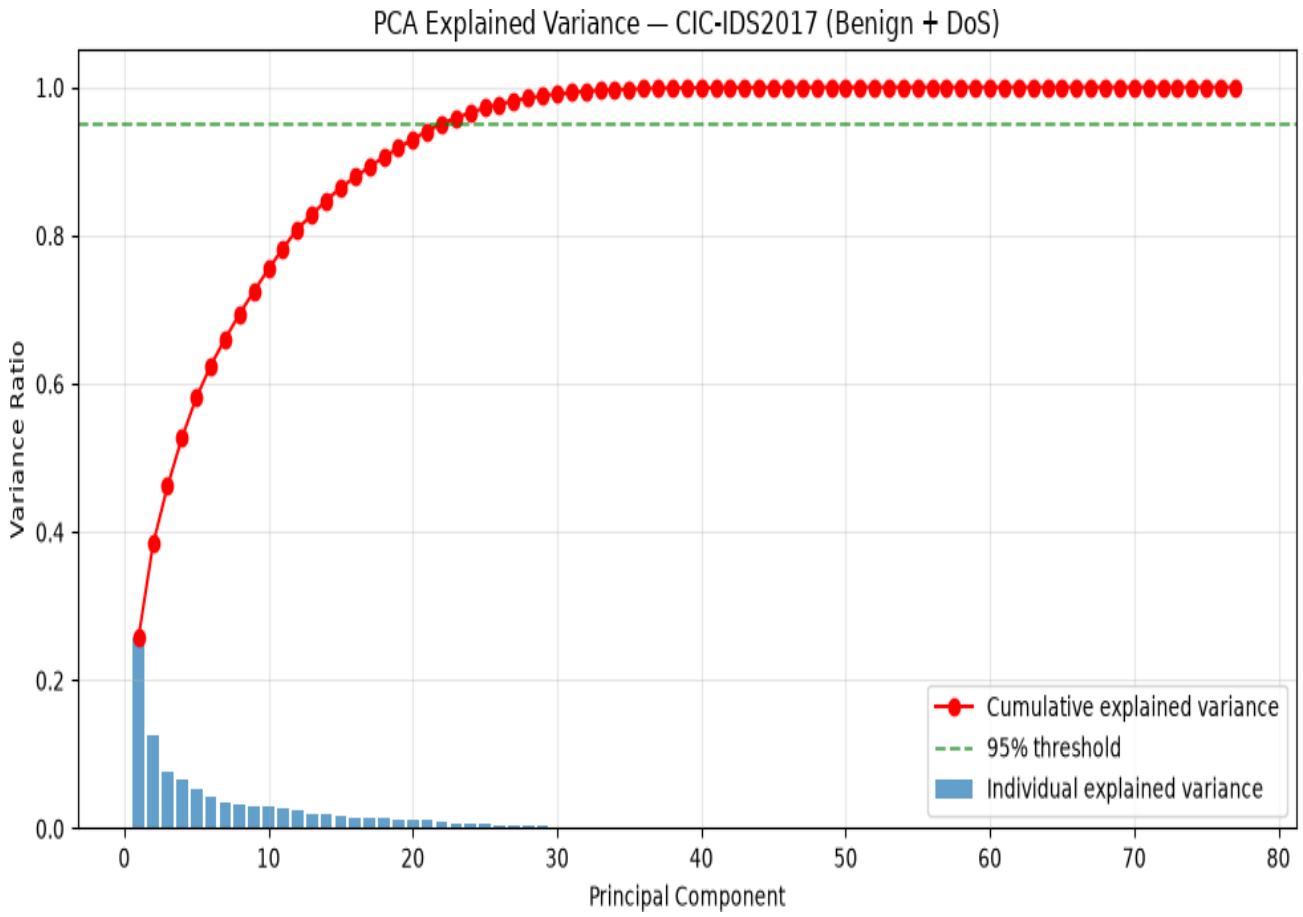
# ------------------
# 4) Apply PCA
# ------------------
# Full PCA first, to inspect how variance is distributed across all components
pca_full = PCA()
pca_full.fit(X_scaled)
explained_var = pca_full.explained_variance_ratio_
cum_var = np.cumsum(explained_var)

# How many components are needed to reach 95% cumulative variance?
n_components_95 = np.argmax(cum_var >= 0.95) + 1
print(f"Number of components to reach 95% variance: {n_components_95} "
      f"(out of {X_scaled.shape[1]} original features)")
```

#output: Number of components to reach 95% variance: 22 (out of 77 original features)

```
# PCA with 2 components, for visualization
pca_2 = PCA(n_components=2)
X_pca_2 = pca_2.fit_transform(X_scaled)

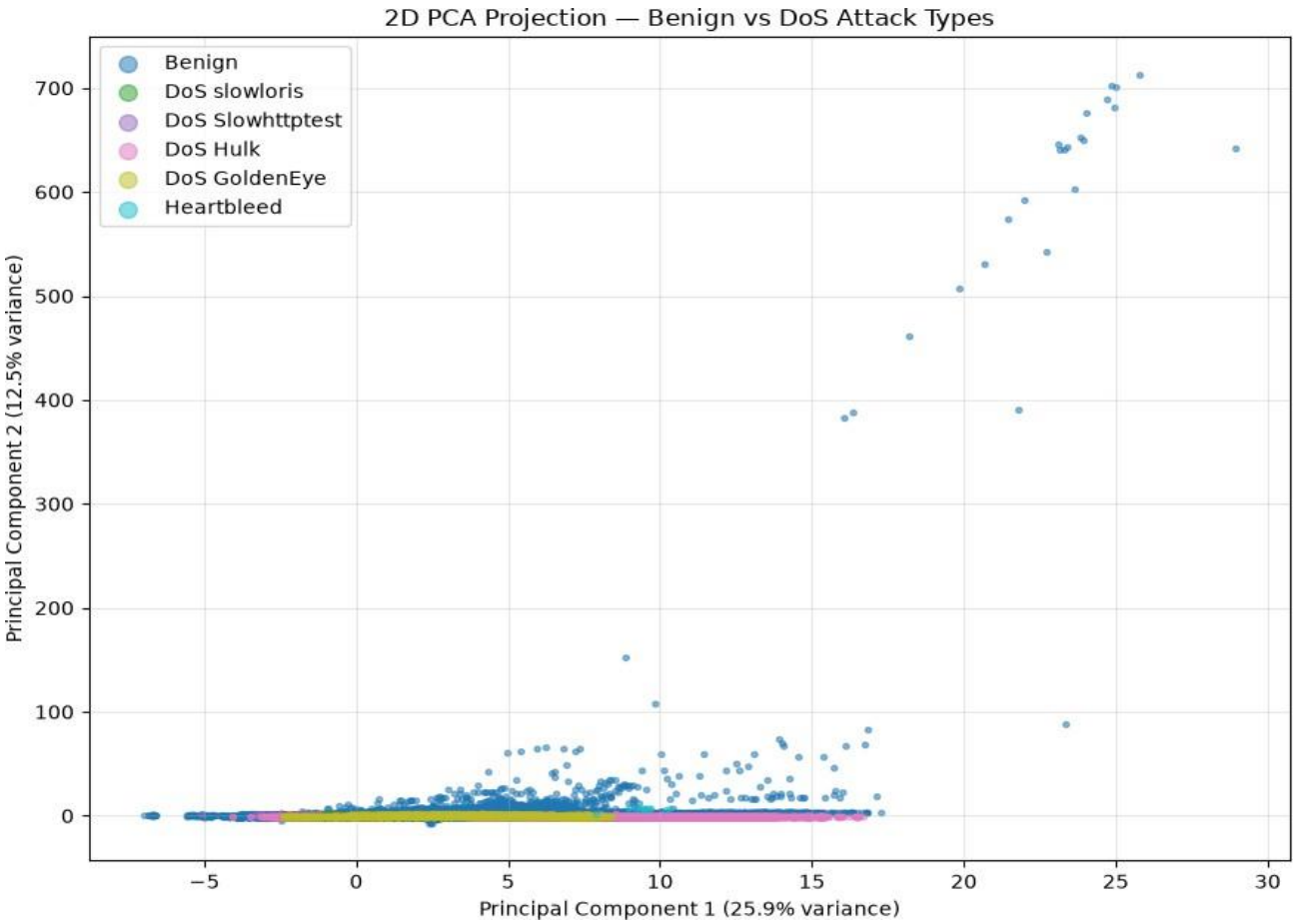
# ------------------
# 5) Plot 1: Explained variance (bar) + cumulative variance (line)
# ------------------
plt.figure(figsize=(10, 5))
plt.bar(range(1, len(explained_var) + 1), explained_var, alpha=0.7,
        label="Individual explained variance")
plt.plot(range(1, len(cum_var) + 1), cum_var, marker="o", color="red",
         label="Cumulative explained variance")
plt.axhline(y=0.95, color="green", linestyle="--", alpha=0.6, label="95% threshold")
plt.xlabel("Principal Component")
plt.ylabel("Variance Ratio")
plt.title("PCA Explained Variance — CIC-IDS2017 (Benign + DoS)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("explained_variance.png", dpi=150)
plt.show()
```



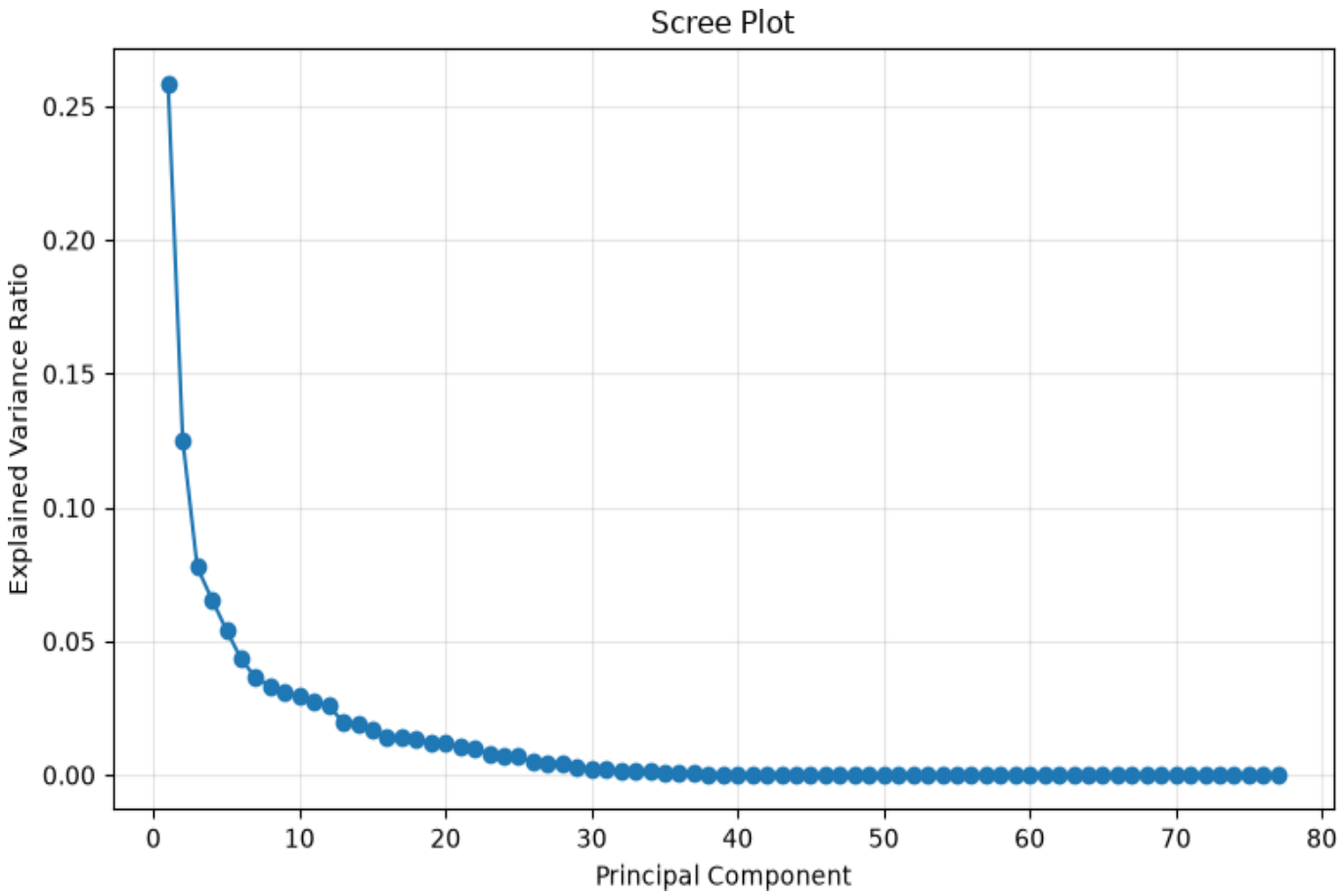

```
# - - - - - - - - - - - - - - - - - - -
# 6) Plot 2: 2D PCA scatter, colored by attack label
# - - - - - - - - - - - - - - - - - - -
plt.figure(figsize=(9, 7))
labels_unique = y.unique()
colors = plt.cm.tab10(np.linspace(0, 1, len(labels_unique)))

for label, color in zip(labels_unique, colors):
    mask = (y == label).values
    plt.scatter(X_pca_2[mask, 0], X_pca_2[mask, 1], s=8, alpha=0.5,
                label=label, color=color)

plt.xlabel(f"Principal Component 1 ({pca_2.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"Principal Component 2 ({pca_2.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.title("2D PCA Projection — Benign vs DoS Attack Types")
plt.legend(markerscale=3, loc="best")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("pca_2d_scatter.png", dpi=150)
plt.show()
```




```
# ------------------------
# 7) Scree plot
# ------------------------
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(explained_var) + 1), explained_var, marker="o")
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance Ratio")
plt.title("Scree Plot")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("scree_plot.png", dpi=150)
plt.show()
```




```
# - - - - - - - - - - - - - - - - - - -
# 8) Save transformed data
# - - - - - - - - - - - - - - - - - - -
pca_df = pd.DataFrame(X_pca_2, columns=["PC1", "PC2"])
pca_df["Label"] = y.values
pca_df.to_csv("pca_transformed_data.csv", index=False)

print("Explained variance ratio for first 2 PCs:", pca_2.explained_variance_ratio_)
print("Sum of first 2 PCs variance:", pca_2.explained_variance_ratio_.sum())
print("Saved: pca_transformed_data.csv, explained_variance.png, pca_2d_scatter.png,
scree_plot.png")


“””output: Explained variance ratio for first 2 PCs: [0.2585246  0.12531631]
Sum of first 2 PCs variance: 0.38384090773893154
Saved: pca_transformed_data.csv, explained_variance.png, pca_2d_scatter.png, scree_plot.png
“””
```